

CLOUD COMPUTING COURSE

Chapter2: Introduction to Compute Virtualization

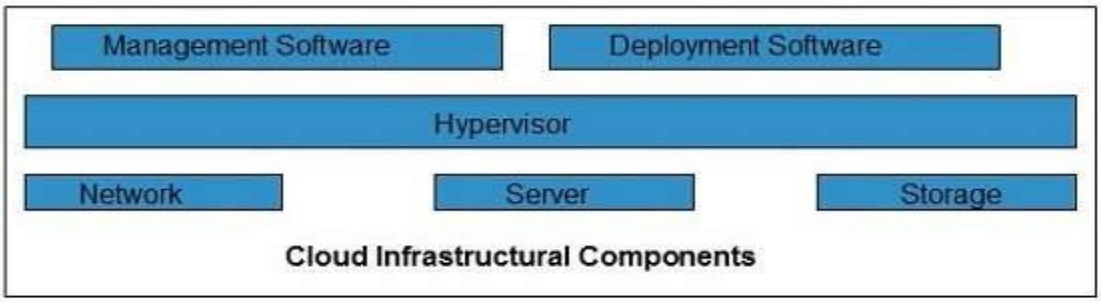
ABSTRACT
Chapter 2 Introduces compute virtualization technologies.

Instructor: Asma'a Khtoom

Cloud Computing Infrastructure

Cloud computing infrastructure is the collection of hardware and software elements needed to enable cloud computing. It includes computing power, networking, and storage, as well as an interface for users to access their virtualized resources. The virtual resources mirror a physical infrastructure, with components like servers, network switches, memory and storage clusters.

Cloud infrastructure consists of servers, storage devices, network, cloud management software, deployment software, and platform virtualization.



1. **Management Software:** It helps to maintain and configure the infrastructure.
2. **Deployment Software:** It helps to deploy and integrate the application on the cloud.
3. **Hypervisor:** It is a low-level program that acts as a Virtual Machine Manager. It allows sharing the single physical instance of cloud resources between several tenants.
4. **Network:** It is the key component of cloud infrastructure. It allows to connect cloud services over the Internet. It is also possible to deliver network as a utility over the Internet.
5. **Server:** The server helps to compute the resource sharing and offers other services such as resource allocation and de-allocation, monitoring the resources, providing security etc.
6. **Storage:** Cloud keeps multiple replicas of storage. If one of the storage resources fails, then it can be extracted from another one, which makes cloud computing more reliable.

Cloud Computing Technologies

The main enabling technology for cloud computing is **virtualization**.

1. Virtualization Definition:

It is the creation of a virtual (rather than actual) version of something, such as an operating system (OS), a server, a storage device or network resources.

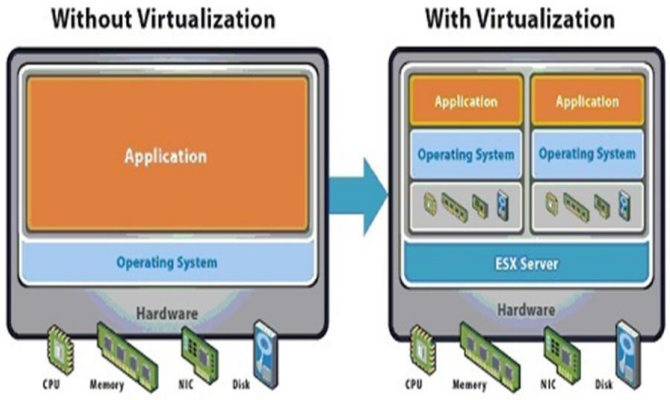
2. Virtualization History:

Virtualization began in the 1960s, as a method of logically dividing the system resources provided by mainframe computers between different applications.

3. Virtualization Benefits

Virtualization uses software that simulates hardware functionality in order to create a virtual system. This practice allows IT organizations to operate multiple operating systems, more than one virtual system and various applications on a single server. The benefits of virtualization include:

- **Reduced IT costs**
- **Efficient resource utilization**
- **Easier testing**
- **Quicker backups**
- **Easier disaster recovery**



4. Virtualization Types:

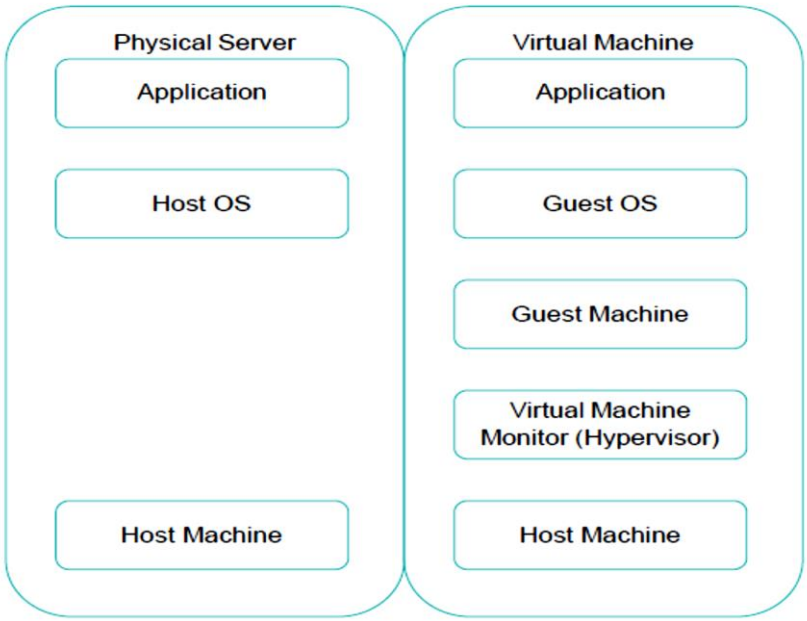
In this course we will cover three types of virtualization

1. **Compute Virtualization**
2. **Network Virtualization**
3. **Storage Virtualization**

5. Virtualization Concept

Creating a virtual machine over existing operating system and hardware is referred as Hardware Virtualization. Virtual Machines provide an environment that is logically separated from the underlying hardware.

The machine on which the virtual machine is created is known as **host machine** and virtual machine is referred as a **guest machine**. This virtual machine is managed by software, which is known as **hypervisor**.



Guest OS:

Operating system running in a virtual machine (VM)

Guest Machine:

Virtual machine created through virtualization

Hypervisor:

Virtualization software layer, or

Virtual Machine Monitor (VMM)

Host OS:

Operating system running in a physical machine

Host Machine:

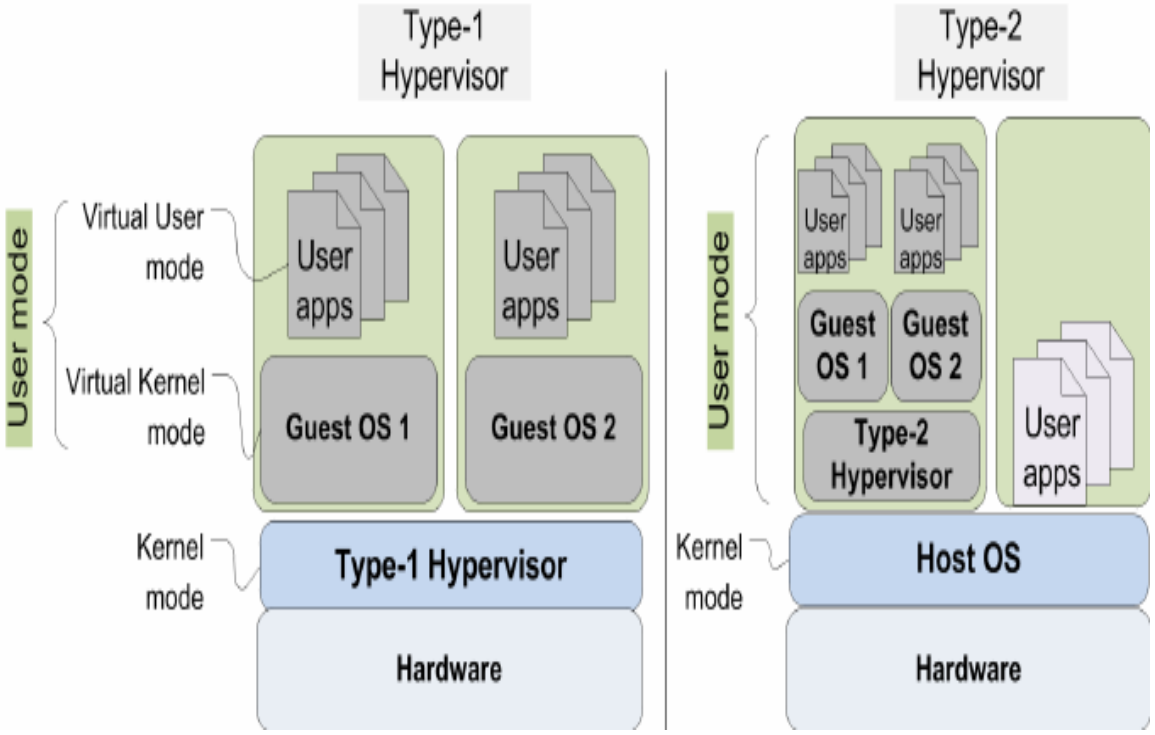
Physical machine

VM Definition: A virtual machine is a computer file, typically called an image that behaves like an actual computer. In other words, creating a computer within a computer. It runs in a window, much like any other program, giving the end user the same experience on a virtual machine, as they would have on the host operating system itself.

While virtualization is converting physical servers into logical folders or files called **VM's,** these folders or files (VM) divided into two parts:

1. VM configuration files
2. User data files

6. Hypervisor Types:



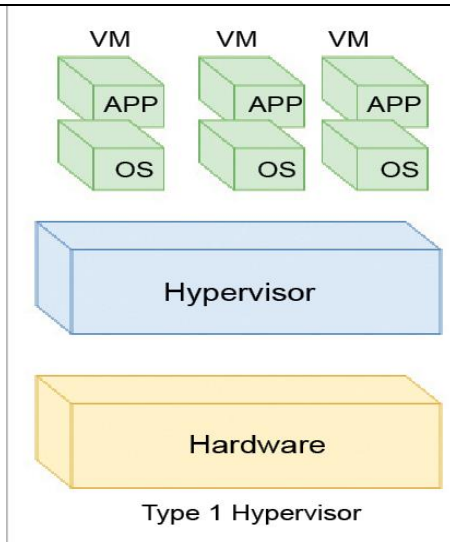
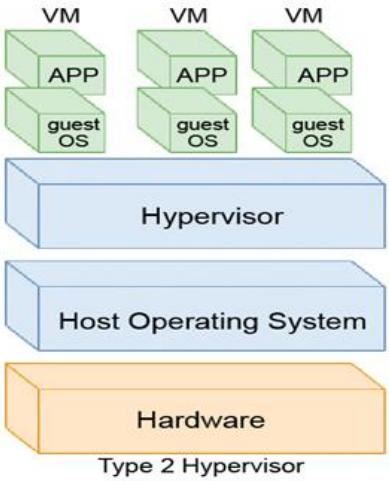
Hypervisors are able to sit on top of an OS or they can be directly installed onto the hardware. This directs us to the different types of hypervisor, which comes in two types:

1. **Type 1** (also called **bare-metal**): The most common hypervisor in **cloud computing**, it is designed to sit directly on underlying hardware and provide the ability to virtualize the hardware resources for use by the virtual machines.

Type1 Hypervisor **Examples: VMware ESXi, Citrix Xen Server , Microsoft Hyper-V.**

2. **Type 2** hypervisor (also **called hosted**): requires a host operating system and it runs on this host machine as another applications and used to create VM's behind the host own applications. Type2 Hypervisor **Examples**: **VirtualBox, Qemu, VMware Workstation** player. Type2 is more often used for **testing and labs and for home usage.**

Comparison between Type1 and Type2

Feature	Type1	Type2
Installation Position	Run directly on the system Hardware.	Run on a host OS.
Support	Hardware Virtualization	OS Virtualization
Security	More secure	Less secure
Performance	Very high	Less than type1
Advantage	VMs can run different types of guest OSs and applications independent of the host OS.	Easy to implement
Disadvantage	Hard Development	Only the applications supported by the host OS can be installed and used
Example	VMware ESXi , XEN Server	KVM, Virtual Box
	 Type 1 Hypervisor	 Type 2 Hypervisor

7. Virtualization Features

There are four main **VM features** supported by both type1 and type2 hypervisors:

1. **Partitioning**
   - Run multiple operating systems on one physical machine.
   - Divide system resources between virtual machines.

2. **Isolation**
   - Provide fault and security isolation at the hardware level.
   - Preserve performance with advanced resource controls.

3. **Encapsulation**
   - Each VM is saved as a group of hardware-independent files.
   - Move and copy virtual machines as easily as moving and copying files, which support VM migration.

4. **Hardware Independence**
   - Provision or migrate any virtual machine to any physical server.
   - The migration of VMs can be successful as long as **the same VMM** running on the **target host** as that on the **source host**.

8. Virtualization vs. Cloud Computing

Virtualization is software that makes computing environments independent of physical infrastructure, while **cloud computing** is a service model that delivers shared computing resources (software and/or data) on demand via the Internet. As complementary solutions, organizations can begin by virtualizing their servers and then moving to cloud computing for even greater agility and self-service.

Virtualization and cloud computing **are not interchangeable.**

9. Compute Virtualization

It is a technique, which allows sharing single physical instance of a resource among multiple organizations or tenants (customers). It does so by assigning a logical name to a physical resource and providing a pointer to that physical resource on demand.

This done by using a software layer called a **hypervisor**. The resources include the **CPU's, memory and input/output (I/O).**

Hypervisors take the physical resources and separate them so they can be utilized by the virtual environment.

1. CPU Virtualization Types

CPU virtualization can be achieved by following techniques:

- **Full virtualization** using binary translation
- **Para virtualization**
- **Hardware assisted virtualization**

1. Full virtualization:

- Hypervisor directly interact with the hardware such as CPU, disks.
- Does dynamic binary translation/ rewriting
- Creates emulation layer to emulate hardware
- Virtual Machine Monitor (Hypervisor) traps and emulates privileged instructions.
- The guest OS is not aware it is being virtualized and requires no modification.
- It offers the best isolation and security for virtual machines.
- It simplifies migration and portability.
- Full virtualization is usually bit slower, because of all emulation.
- **Examples- Hyper-V, VMware vSphere, QEMU.**

2. Para virtualization:

- The guest operating system is aware that it is a guest.
- Guest OS is modified to process privileged instructions.
- Hypervisor does not need large amounts of processing power to manage guest OS.
- VMM is no longer required to translate instructions; it is only responsible for handling virtualization layer.
- Guest OS uses specialized API to talk with VMM.
- The performance of Para virtualization is more than full virtualization
- Para virtualization cannot support unmodified operating systems (e.g. Windows 2000/XP), its compatibility and portability is poor.
- **Example- Xen**

3. Hardware-Assisted virtualization:

- The VMM uses processor extension (Intel-VTx or AMD-V) to intercept and emulate privileged instructions.
- Guest OS runs at kernel level (Ring 0) and VMM runs in more privileged Ring than the Ring 0.
- Hardware-assisted virtualization is used to solve virtualization vulnerabilities, simplify VMM software, and eliminate the need for Para virtualization or binary translation.

2. Memory Virtualization

Memory virtualization decouples volatile random access memory (RAM) resources from individual systems in the data center and then aggregates those resources into a virtualized memory pool available to any computer in the cluster.

Benefits of Memory Virtualization: Applications can take advantage of a very large amount of memory to:

1. Improve overall performance.
2. Improve system utilization.
3. Increase memory usage efficiency.

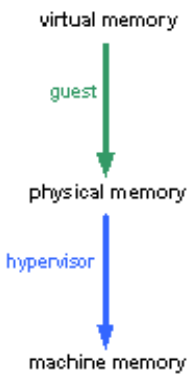
Memory addresses in Virtualization

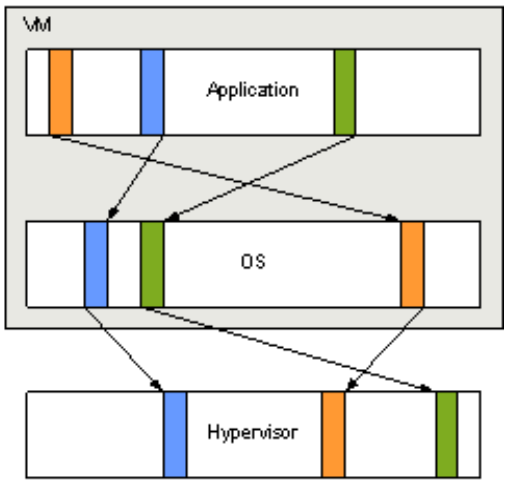
- **Virtual memory address**: presented to applications by the guest OS.
- **Physical memory address**: presented to the VM by Hypervisor.
- **Machine memory address**: provides a contiguous, zero-based, addressable memory space for use by the VM.

Address Binding is the process of mapping from one address space to another address space.

1. In the virtual machine, the **guest operating system's** page tables maintain the mapping from **virtual memory (VA)** to **physical memory (PA)**. **(VA→PA)**
2. The **Hypervisor** then mapping from the **physical memory (PA)** to the machine memory **(MA)** on the underlying machine. **(PA→MA)**

- Each virtual machine sees a contiguous, zero-based, addressable physical memory space.
- The underlying machine memory on the server used by each virtual machine is not necessarily contiguous.





3. I/O Virtualization

With compute virtualization, a large number of VMs can be created on a single host, and the VMs all need to access the I/O devices of this host. However, I/O devices are limited. I/O device sharing among multiple VMs requires VMM.

VMM intercepts access requests from VMs to I/O devices, simulates I/O devices using software, and responds to I/O requests.

This way, **multiple VMs can access I/O resources concurrently.**

I/O virtualization can be implemented in the following methods:

1. Full virtualization
2. Paravirtualization
3. Hardware-assisted virtualization. Hardware-assisted virtualization is the mainstream technology for I/O virtualization.

1. Full virtualization

The Mechanism

1. VMM virtualizes I/O devices for VMs. When a VM initiates an I/O request to an I/O device, VMM intercepts the request sent by the VM.
2. Then sends the real access request to the physical device for processing.

Full Virtualization Characteristics

- No matter which type of OS is used by the VM, the OS does not need to be modified for I/O virtualization.
- Multiple VMs can directly use the I/O device of the physical server.
- However, VMM needs to intercept I/O requests delivered by each VM in real time and emulates the request to a real I/O device, which causes severe **performance loss** to the server.

2. Para virtualization

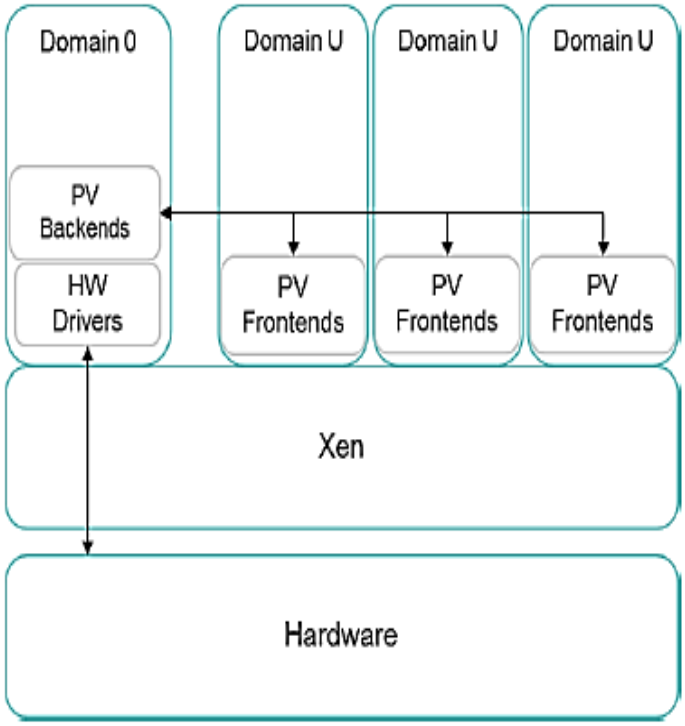
Unlike full virtualization, paravirtualization needs **a privileged VM**.

The Mechanism

1. Paravirtualization requires each VM to run a **frontend driver**.
2. When VMs need to access an I/O device, the VMs send I/O requests to the **privileged VM** through the frontend driver.
3. The backend driver of the privileged VM collects the I/O request sent by each VM.
4. Then, the backend driver processes multiple I/O requests by time and by channel.
5. The privileged VM runs the physical I/O device driver and sends the I/O request to the physical I/O device.
6. After processing the request, the I/O device returns the processing result to the privileged VM.

Para Virtualization Characteristics

- VMs send I/O requests to a privileged VM and then the privileged VM accesses a real I/O device. **This reduces the performance loss of VMM.**
- The VM OS needs to be modified. Specifically, the I/O request processing method of the OS needs to be changed so that all the I/O requests can be sent to the privileged VM for processing.
- This type can be used only **with open source OS's like Linux.**



3. Hardware-assisted virtualization

The Mechanism

Hardware-assisted **virtualization directly installs the I/O device driver in** the VM OS without any change to the OS.

Hardware-assisted Virtualization Characteristics

- This method is equivalent to traditional PC OS access to hardware. Therefore, the time required for a VM to access the I/O hardware is the same as that for a traditional PC to access the I/O hardware.
- Hardware-assisted virtualization **outperforms** full virtualization and paravirtualization in terms of I/O performance.
- However, hardware-assisted virtualization requires special hardware support.

10. Mainstream Compute Virtualization Technologies

In cloud computing, we mainly use virtualization to implement **IaaS** cloud services. There are many mainstream virtualization technologies, generally:

Open-Source	Closed-source
Are free of charge and can be used anytime.	Are generally not free of charge and can be used out of the box.
Users can customize some special requirements based on open-source code	Users cannot view or customize source code.
Once a problem occurs in the system, the system recovery strongly relies on the administrator's skillset and experience.	If a system problem occurs, vendors provide all-round support.
Include KVM and Xen.	Include Microsoft Hyper-V, VMware vSphere, and Huawei FusionSphere.

KVM vs Xen

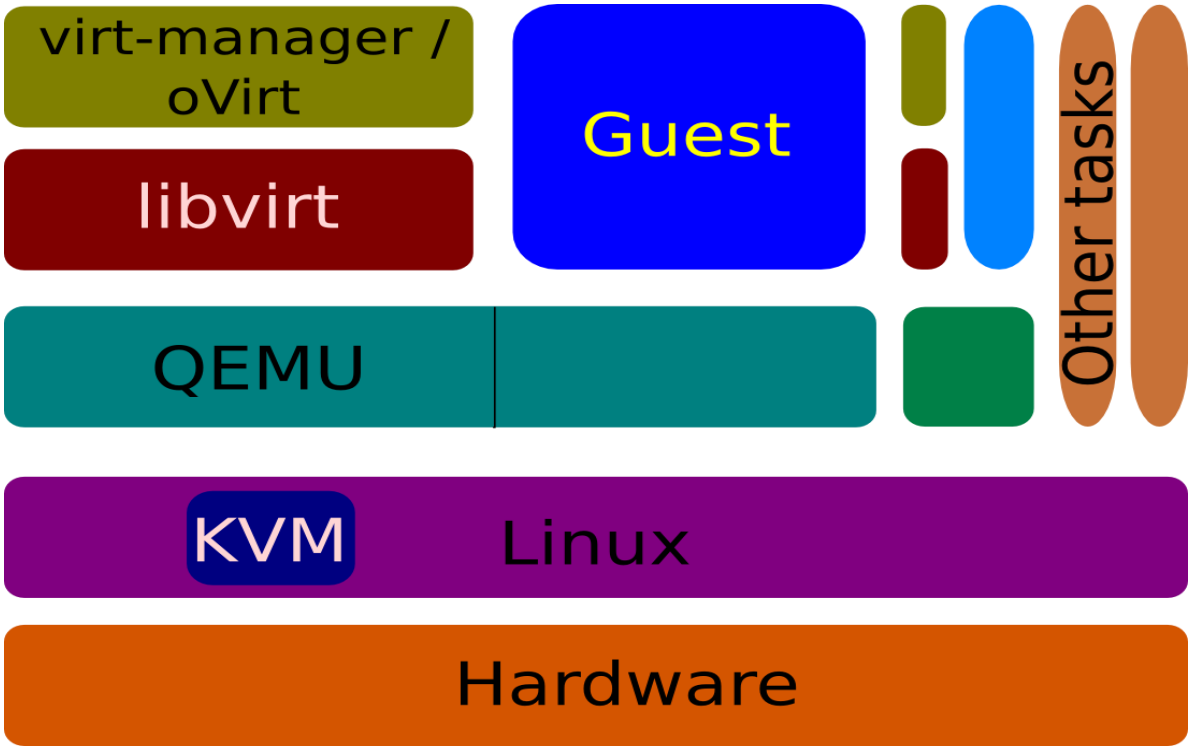
Open Source Technology Software's	
KVM	**Xen**
KVM is full virtualization	Supports both paravirtualization and full virtualization.
It is **Type2 Hypervisor**, KVM, a module in the Linux kernel.	It is **Type1 Hypervisor** it is directly runs on hardware, and VMs run on Xen.
VMs Type on it: Common VM's	VMs Type on it: • Privileged VM • Common VM's
- It is used to virtualize CPUs and memory. - I/O devices (such as NICs and disks) need to be virtualized by QEMU.	- It is used to virtualize all compute resources.

11. Virtualization Products

1. KVM Virtualization Product

Huawei virtualization products earlier than the 6.3 version are developed based on Xen. In 6.3 and later, they are developed based on Kernel-based Virtual Machine (KVM).

KVM is a **Type-II full virtualization** solution. It is a **Linux** kernel module. A physical machine with a Linux kernel module installed can function as a hypervisor, which does not affect the other applications running on the Linux OS.



After the KVM module is installed in a common Linux OS, three running modes are added:

1. **Guest Mode:** VMs, including their CPUs, memory, and disks.
2. **User Mode:** The quick emulator (QEMU) typically runs in this mode. QEMU emulates I/O requests.
3. **Kernel Mode**: In this mode, the hardware can be operated. When the guest OS executes an I/O operation or privileged instruction, a request needs to be submitted to the user mode, and then the user mode initiates a hardware operation request to the kernel mode again to operate the hardware.

A KVM architecture consists of three parts:

1. **KVM kernel module:**

   The KVM kernel module is the core of a KVM VM. This module initializes the CPU hardware, enables the virtualization mode, runs the guest machine in the Guest mode, and supports the running of the virtual client.

- **KVM runs in the kernel space.**
- **Only provides CPU and memory virtualization.**

2. QEMU:

A VM requires other I/O devices such as NICs and hard disks besides CPUs and memory. QEMU is required to implement NIC and hard disk virtualization functions.

QEMU **was not a part of KVM kernel**. It was a universal **open-source** virtualization emulator that uses pure software to implement virtualization. The guest OS considers that it is interacting with hardware. Actually, QEMU is interacting with hardware. This means that all interactions with the hardware need to pass through QEMU. Therefore, the simulation **performance delivered by QEMU is low**.

- **QEMU runs in the user space**

3. Management tool.

In addition to virtualization of various devices, QEMU-KVM provides native tools for creating, modifying, and deleting VMs. However, **Libvirt** is the most widely used tool and API for managing KVM VMs.

Libvirt is:

1. An **open-source** project and is a powerful management tool. It is able to manage virtualization platforms such as KVM, Xen, VMware, and Hyper-V.
2. Libvirt is an API developed using the **C language**.
3. In cloud computing, there are various hypervisors. Each hypervisor has its own management tool, and parameters are complex and difficult to use. Hypervisors **are not unified**, and there is no unified **programming interface** to manage them, which severely affects the cloud computing environment. With **Libvirt**, it can connect to various hypervisors, such as KVM and Xen, and provide APIs in various languages.
4. Libvirt serves as the **middle layer** between the management tool and hypervisor and is completely transparent to upper-layer users.

2. FusionCompute Virtualization Product

It virtualizes server, storage, and network resources, providing elastic resource pools for automatic resource scheduling and management. It consists of Compute Node Agent (CNA), a virtualization engine, and Virtual Resource Management (VRM).

It is based on **the Linux OS.**

Benefit of FusionCompute

Following benefits for customers:

1. Improves infrastructure resource utilization data centers.
2. Significantly accelerates service rollout.
3. Substantially reduces power consumption in data centers.
4. Provides rapid automatic fault recovery for services, decreases data center costs, and increases system runtime by leveraging high availability and powerful restoration capabilities of virtualized infrastructure.

FusionCompute Parts

FusionCompute consists of two parts:

1. **Computing Node Agent (CNA)**

   CNA manages VMs and resources on the local node (the server), it provides the virtualization function, and it based on the Linux OS.

CNA provides the following functions:

1. Provides the virtual computing function.
2. Manages the VMs running on compute nodes.
3. Manages compute, storage, and network resources on compute nodes.

2. **Virtual Resource Manager (VRM):**

VRM is the resource scheduling and management software. VRM manages **clusters** or resources in the resource pool. It functions as **the management tool** of KVM.

Administrators and common users can manage and use FusionCompute on the GUI-based portal of VRM.

VRM provides the following functions:

1. Manages block storage resources in a cluster.
2. Manages network resources, such as IP addresses and virtual local area network (VLAN) IDs, in a cluster and allocates IP addresses to VMs.
3. Manages the lifecycle of VMs in a cluster, distributes, and migrates VMs across compute nodes.
4. Dynamically scales resources in a cluster.
5. Implements centralized management of virtual resources and user data and provides elastic computing, storage, and IP address services.

